

REST Assured + API Automation with Java — Complete Interview Preparation Guide

1. API Testing Fundamentals

What is an API?

An API (Application Programming Interface) allows two software systems to communicate with each other.

Example:

- Mobile App → calls API → Backend Server
- Frontend UI → calls API → Database via backend

2. What is API Testing?

API Testing validates:

- Functionality
- Reliability
- Security
- Performance
- Data correctness
- Business logic

without using the UI.

3. Why API Testing is Important

Advantages

- Faster than UI testing
- Stable (less flaky)
- Independent of frontend changes
- Detects backend issues early
- Better test coverage
- Easy automation

- Supports CI/CD
-

4. Types of APIs

REST API

Most commonly used.

Uses:

- HTTP methods
 - JSON
 - Stateless communication
-

SOAP API

Older enterprise protocol.

Uses:

- XML
- WSDL
- Strict contracts

Difference:

REST	SOAP
Lightweight	Heavy
JSON/XML	XML only
Faster	Slower
Easy	Complex
Stateless	Can be stateful

5. REST Principles

REST = Representational State Transfer

REST Constraints

Stateless

Server does not store client session.

Each request must contain complete information.

Client-Server Architecture

Frontend and backend separated.

Cacheable

Responses can be cached.

Uniform Interface

Standard HTTP methods used.

6. HTTP Basics

HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create resource
PUT	Full update
PATCH	Partial update
DELETE	Remove resource

7. Difference Between PUT and PATCH

PUT	PATCH
Full update	Partial update
Replaces entire object	Updates only changed fields
Idempotent	Usually idempotent

8. Idempotent Methods

Same request multiple times → same result.

Idempotent

- GET
- PUT
- DELETE

Non-idempotent

- POST

Interview Question: Why is POST non-idempotent? Because each request may create a new resource.

9. HTTP Status Codes

1xx → Informational

- 100 Continue

2xx → Success

- 200 OK
- 201 Created
- 204 No Content

3xx → Redirection

- 301 Moved Permanently
- 302 Found

4xx → Client Error

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 405 Method Not Allowed
- 409 Conflict

5xx → Server Error

- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable

10. Difference Between 401 and 403

401	403
Authentication failed	Authenticated but no permission
Invalid credentials	Access denied

11. Headers in API

Metadata sent with requests/responses.

Examples:

- Content-Type
- Authorization
- Accept
- Cache-Control

12. Content-Type vs Accept

Content-Type	Accept
Format of request body	Expected response format

Example:

Content-Type: application/json Accept: application/json

13. Common Content Types

- application/json
 - application/xml
 - multipart/form-data
 - text/plain
-

14. Authentication Types

Basic Authentication

Username + Password encoded in Base64.

Bearer Token

JWT/OAuth token.

Authorization: Bearer token

OAuth 2.0

Industry standard authorization framework.

API Key

Simple authentication using unique key.

15. JWT Token

JSON Web Token.

Contains:

- Header
- Payload
- Signature

Used for:

- Authentication
 - Authorization
-

16. API Request Components

- URL
 - Endpoint
 - Method
 - Headers
 - Query Params
 - Path Params
 - Body
 - Authentication
-

17. Query Parameter vs Path Parameter

Query Param	Path Param
Optional/filter data	Identifies resource
?id=10	/users/10

18. Request Body Types

JSON

Most common.

XML

SOAP APIs.

Form Data

Used in file uploads.

19. What is JSON?

JavaScript Object Notation.

Lightweight data exchange format.

20. JSON Data Types

- String
 - Number
 - Boolean
 - Object
 - Array
 - Null
-

21. JSON Object vs Array

Object	Array
Key-value pairs	Ordered list
{ }	[]

22. API Testing Scenarios

Positive Testing

Valid inputs.

Negative Testing

Invalid inputs.

Examples:

- Invalid token
 - Missing fields
 - Wrong data types
 - Boundary values
-

23. Important API Validations

- Status code validation
 - Response body validation
 - Schema validation
 - Header validation
 - Response time validation
 - Database validation
-

24. What is Schema Validation?

Validates structure of response JSON.

Checks:

- Mandatory fields
 - Data types
 - Structure
-

25. What is Contract Testing?

Validates agreement between:

- Consumer
- Provider

Ensures API response format remains unchanged.

Popular tool:

- Pact
-

26. What is Serialization?

Java Object → JSON

27. What is Deserialization?

JSON → Java Object

28. POJO Class

Plain Old Java Object.

Used for:

- Serialization
 - Deserialization
-

29. REST Assured Introduction

What is REST Assured?

Java library used for API automation testing.

Supports:

- REST APIs
 - JSON/XML validation
 - Authentication
 - Schema validation
-

30. Why REST Assured?

- Easy syntax
- BDD style
- Integrates with TestNG/JUnit
- Supports JSONPath/XMLPath
- Strong validations

31. REST Assured Architecture

Main classes:

- RequestSpecification
- Response
- ValidatableResponse

Flow:

given().when().then()

32. BDD Style in REST Assured

given().header().body()

.when().post()

.then().statusCode(201);

33. REST Assured HTTP Methods

get() post() put() patch() delete()

34. Common REST Assured Methods

Method	Purpose
given()	Precondition
when()	Action
then()	Validation
body()	Request body
header()	Add headers
param()	Add params

35. How to Send Headers

```
given().header("Authorization","Bearer token")
```

36. How to Send Query Parameters

```
queryParams("id",101)
```

37. How to Send Path Parameters

```
pathParam("id",101)
```

38. How to Validate Response

```
.then().statusCode(200).body("name", equalTo("John"));
```

39. Hamcrest Matchers

Used for assertions.

Examples:

- equalTo()
 - containsString()
 - hasItems()
 - notNullValue()
-

40. JSONPath

Used to extract values from JSON.

Example: `response.jsonPath().getString("name");`

41. Difference Between JSONPath and XPath

JSONPath	XPath
JSON	XML
Faster	Complex

42. Response Extraction

```
Response response = given().when().get();
```

```
String name = response.jsonPath().getString("name");
```

43. Request Specification

Reusable request configuration.

```
Example: RequestSpecification req = new RequestSpecBuilder().setBaseUrl("https://api.com").build();
```

44. Response Specification

Reusable response validation.

45. Base URI vs Base Path

Base URI	Base Path
Domain	Endpoint path

46. Serialization Example

```
User user = new User(); user.setName("Rakesh");
```

```
given().body(user)
```

47. Deserialization Example

```
User user = response.as(User.class);
```

48. File Upload in REST Assured

```
multiPart("file",new File("test.pdf"))
```

49. File Download Validation

Validate:

- Status code
 - File existence
 - Content type
-

50. Cookies in API

Stored client-side session data.

51. Session Handling

```
sessionId("abc123")
```

52. OAuth2 in REST Assured

```
auth().oauth2(token)
```

53. Basic Auth Example

```
auth().basic("user","pass")
```

54. Logging in REST Assured

`log().all()`

Used for:

- Debugging
 - Request/response visibility
-

55. Response Time Validation

`time(lessThan(2000L))`

56. Schema Validation Example

`matchesJsonSchemaInClasspath("schema.json")`

57. TestNG in API Automation

Features:

- Annotations
 - Parallel execution
 - Grouping
 - DataProvider
 - Reporting
-

58. Important TestNG Annotations

`@BeforeSuite @BeforeClass @BeforeMethod @Test @AfterMethod`

59. DataProvider

Used for data-driven testing.

60. API Chaining

Using response from one API in another API.

Example:

- Create user
 - Extract userId
 - Pass userId to GET API
-

61. CRUD Operations

Operation	Method
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

62. API Versioning

Methods:

- URI versioning /v1/users
 - Header versioning
-

63. Pagination

Used for large datasets.

Parameters:

- page
 - limit
 - offset
-

64. Rate Limiting

Restricts number of requests.

Response:

- 429 Too Many Requests
-

65. Mocking APIs

Used when backend unavailable.

Tools:

- WireMock
 - Postman Mock Server
-

66. API Testing Tools

Popular tools:

- REST Assured
 - Postman
 - Swagger
 - SoapUI
 - JMeter
-

67. Swagger / OpenAPI

API documentation framework.

Contains:

- Endpoints
 - Request/response models
 - Auth details
-

68. Postman vs REST Assured

Postman	REST Assured
Manual + automation	Automation
GUI based	Code based
Easy for beginners	Better for frameworks

69. SOAP vs REST Interview Answer

SOAP:

- Protocol
- XML
- Secure
- Enterprise

REST:

- Architectural style
 - JSON
 - Faster
 - Lightweight
-

70. REST Assured Framework Architecture

Common Framework Layers

Base Layer

- Base URI
- Common methods

Utilities

- JSON utilities
- Excel utilities

Test Layer

- Test cases

Reporting Layer

- Extent Reports

Config Layer

- Environment configs
-

71. Maven in API Automation

Dependency management tool.

72. Important Maven Lifecycle Phases

clean compile test package install

73. Common Dependencies

- REST Assured
 - TestNG
 - Jackson/Gson
 - JsonSchemaValidator
-

74. Jackson vs Gson

Both used for JSON parsing.

Jackson	Gson
Faster	Simpler
More powerful	Lightweight

75. CI/CD Integration

Tools:

- Jenkins
- GitHub Actions
- GitLab CI

Used for:

- Automated execution
 - Regression testing
-

76. Parallel Execution

Run multiple tests simultaneously.

Benefits:

- Faster execution
-

77. Environment Management

Environments:

- Dev
- QA
- UAT
- Prod

Handled using:

- config.properties
-

78. Sensitive Data Handling

Never hardcode:

- Passwords
- Tokens

Use:

- Environment variables
 - Jenkins credentials
-

79. Retry Mechanism

Used for transient failures.

Example:

- 503 temporary server issue
-

80. Common Interview Scenario Questions

Q: How do you validate dynamic response data?

Answer:

- Extract response
 - Store values
 - Use assertions
-

Q: How do you handle token expiration?

Answer:

- Generate token dynamically
 - Refresh token when expired
-

Q: How do you handle environment switching?

Answer:

- Property/config files
 - Maven profiles
-

Q: How do you validate nested JSON?

Using JSONPath.

Example: `data[0].employee.name`

81. API Security Testing Concepts

Check:

- Unauthorized access
 - SQL injection
 - Broken authentication
 - Sensitive data exposure
-

82. Performance Testing Basics

Validate:

- Response time
- Throughput
- Concurrent users

Tools:

- JMeter
 - Gatling
-

83. Difference Between API Testing and UI Testing

API Testing	UI Testing
Fast	Slow
Stable	Flaky
Backend	Frontend
Early testing	End-to-end

84. Smoke vs Sanity vs Regression

Smoke	Sanity	Regression
Basic validation	Small validation	Full validation

85. What Happens Internally When API Request is Sent?

Flow: 1. Client sends HTTP request 2. DNS lookup 3. Server receives request 4. Authentication 5. Business logic execution 6. DB interaction 7. Response generated

86. Difference Between Monolithic and Microservices APIs

Monolith	Microservices
Single application	Multiple services
Tight coupling	Loose coupling
Hard scaling	Easy scaling

87. API Automation Best Practices

- Use reusable methods
 - Avoid hardcoding
 - Proper logging
 - Schema validation
 - Parameterization
 - Good assertions
 - CI integration
-

88. Common Real-Time API Challenges

- Dynamic tokens

- Unstable test data
 - Rate limits
 - Environment issues
 - Third-party dependency failures
-

89. Most Asked REST Assured Interview Questions

Explain given-when-then.

BDD syntax:

- given → preconditions
 - when → action
 - then → validation
-

How do you perform serialization?

Convert Java object to JSON using Jackson/Gson.

How do you perform API chaining?

Extract response and pass values into next API.

How do you validate schema?

Using JSON schema validator library.

How do you manage reusable components?

Using Base classes and Request Specifications.

90. Advanced Topics (Very Important)

Filters in REST Assured

Intercept requests/responses.

Used for:

- Logging
 - Custom validations
-

Interceptors

Custom request handling.

SSL Validation

`relaxedHTTPSValidation()`

Proxy Configuration

Used in network debugging.

Connection Timeout Handling

Important for stability.

91. Real-Time Framework Structure

`src/test/java` |—— tests |—— base |—— utilities |—— payloads |—— pojo |—— listeners

`src/test/resources` |—— config |—— schemas

92. Database Validation in API Testing

Validate backend data using SQL queries.

Used for:

- End-to-end verification
-

93. Assertions in API Testing

Validate:

- Status
 - Headers
 - Body
 - DB data
-

94. Soft Assert vs Hard Assert

Hard Assert	Soft Assert
Stops execution	Continues execution

95. Common Defects Found in APIs

- Wrong status codes
 - Missing fields
 - Null responses
 - Incorrect auth
 - Slow responses
 - Data corruption
-

96. HTTP vs HTTPS

HTTP	HTTPS
Not secure	Secure
No encryption	SSL/TLS encryption

97. Caching

Improves performance.

Headers:

- Cache-Control
 - ETag
-

98. What is CORS?

Cross-Origin Resource Sharing.

Controls browser access between domains.

99. API Lifecycle

1. Design
 2. Development
 3. Testing
 4. Deployment
 5. Monitoring
 6. Versioning
-

100. Final Interview Preparation Strategy

Focus Areas

Must Strong Topics

- HTTP methods
 - Status codes
 - REST principles
 - REST Assured syntax
 - JSONPath
 - Authentication
 - Serialization/Deserialization
 - Framework design
 - TestNG
 - API chaining
 - Schema validation
-

Frequently Asked Practical Coding Areas

- GET API validation
 - POST request creation
 - Dynamic token handling
 - Query/path params
 - JSON parsing
 - POJO classes
 - Reusable methods
 - DataProviders
 - CRUD automation
-

Senior-Level Questions

How do you design an API framework?

Explain:

- Layered architecture
 - Reusability
 - Config management
 - Reporting
 - Logging
 - CI/CD
-

How do you improve API automation stability?

- Retry mechanism
 - Dynamic waits
 - Stable test data
 - Mock services
 - Better assertions
-

Important Keywords for Interviews

- Stateless
- Idempotent
- Serialization
- Deserialization
- JSON Schema

- OAuth2
- JWT
- API Chaining
- Request Specification
- Hamcrest Matchers
- POJO
- Contract Testing
- Microservices